

AA-07

Fleet Portal Specification

Twenty-four screens for delivery companies: offers, riders, POD, COD remittance

30 August 2026 · Rev 1.0 — baseline

SCREENS	PERMISSIONS	ROLES PER ORG
24	15	4
STEP-UP GATED	FLEET ENDPOINTS	MINIMUM VIEWPORT
3	37	360 px

This document specifies the delivery-company self-serve portal at `fleet.atavishaala.in`: all 24 screens, the 4 roles inside a delivery organisation, the offer auction the whole product is built around, and the single endpoint in the entire programme that releases a customer address. The reader it is written for is FE-2, who builds it, and BE-2, who builds the allocator and the proof-of-delivery path behind it. The reader it is written about is an employee of a third-party delivery company: a dispatcher at a desk, or a rider on a phone. That person is the least trusted actor class in the programme, and section 7.1 explains why in three sentences. It assumes AA-00 sections 0.2 and 0.7, AA-02 for tenancy and the cron that expires offer waves, AA-03 for the job and shipment state machines and the `pii_disclosures` register, AA-04 sections 4.5 and 4.6 for the endpoint behind every action, AA-05 for the admin exception board that receives what this portal fails to absorb, AA-08 for the disclosure regime, AA-09 for the runbooks, and AA-11 for the week each screen lands in.

7.1 WHAT THE FLEET PORTAL IS FOR

Four jobs: take work, execute it, prove it, settle the cash

A delivery company logs into fleet.atavishaala.in to do four things, and nothing else. It takes work from an offer auction it cannot see the inside of. It executes that work by attaching a rider and driving a job through its state machine. It proves the work happened, because an unproven delivery is a dispute the platform loses. And it settles cash, because most of these orders are collected in notes at a door. Every screen in this document belongs to exactly one of those four jobs, and any screen that belonged to none of them was cut.

The design premise is narrower than the screen count suggests. A dispatcher looks at an incoming offer and decides in under ten seconds whether to take it, on the strength of four numbers: how far, how heavy, how much cash to carry, and what it pays. The offer inbox is therefore the whole product and everything else in these 24 screens is bookkeeping around it. That ranking is not rhetoric; it determines what gets built first (AA-11, W5), what gets polished (FL-04 and FL-05), and what ships as a plain table with no empty-state art.

It matters just as much what this portal refuses to be. The four rows below are not future work that slipped; each is a decision with a named reason, and none of them will be reconsidered inside these ten weeks.

JOB	WHAT IT MEANS IN THE PORTAL	SCREENS	WHAT BREAKS IF IT IS SLOW
Take work	A broadcast offer arrives with a countdown. Accept it before another fleet does, or decline it with a reason code. The only surface in the programme where one tenant competes with another.	FL-04, FL-05	The job exhausts all three waves in about 35 minutes, goes UNASSIGNED and lands on the admin exception board (AA-05 section 5.4).
Execute it	Attach a rider and a vehicle, scan at the supplier gate, drive the job through six fleet-owned transitions, and open the manifest to find the door.	FL-06 to FL-12	promised_by is breached. The sla-watch cron raises an advisory alert every ten minutes and ops starts calling.
Prove it	Delivery OTP, a geo point inside the fence, a receiver name and a photo. No pod_records row means the job cannot reach DELIVERED at all.	FL-09, FL-10	The delivery is unprovable, a return dispute has nothing to adjudicate against, and the earnings line for that job is held.
Settle the cash	Declare the COD collected for the period, record the deposit reference, and read the append-only ledger the fleet is paid from.	FL-19 to FL-22	A negative fleet_ledger balance auto-blocks the fleet from every wave, so the first symptom the fleet notices is silence in the offer inbox.

What it is not

THIS PORTAL IS NOT	WHY NOT, AND WHAT STANDS IN ITS PLACE
A route optimiser	Sequencing is the dispatcher's job, and their knowledge of their own city beats anything we would ship in ten weeks with no map budget. The rider stack on FL-11 is ordered by promised_by and stops there. If a dispatcher wants a different order they reassign, which is one action.
A live map	Continuous location ingest and map tiles are both billed, so both are named paid gaps rather than quietly missing features. The programme captures exactly one geo point per delivery attempt, at POD, which is all the 200 m fence needs and all the dispute record needs.
A window onto the marketplace	A fleet sees jobs offered to it and nothing else: no order, no basket contents, no supplier margin, no other fleet's acceptance rate, no idea how many fleets are in the same wave. It bids into a market it cannot observe, and section 7.9 lists the mechanism that enforces each blind spot.
A native application	The rider surface is a responsive web view at a 360 px floor, served from the same Cloudflare Pages build as the dispatcher surface. There is no store listing, no background service and no push. Section 7.8 states what that costs rather than pretending it is a design choice.

WHY THE FLEET IS THE LEAST TRUSTED ACTOR CLASS IN THE PROGRAMME

Three sentences, and every access decision in this document follows from them. Anyone with a GSTIN and a transport licence can register a delivery company, so the population of fleet users is not vetted the way five named staff accounts are. A fleet employee physically holds customer cash, which makes the money surface adversarial rather than merely careful. And the rider is the only principal in the entire programme who is ever shown a customer address, which makes this the only portal where a PII disclosure is a normal part of the working day rather than an incident. Read every subsequent section against those three facts: masked by default, one narrow exception, logged in the same transaction, and a ledger that blocks the actor before a human notices.

7.2 ROLES INSIDE A FLEET ORGANISATION

4 seats, 15 permissions, one of them mobile-only

A fleet organisation has 4 seats. There is no role editor and there will not be one: C-4 cut custom roles across the whole programme, on the grounds that the nine system roles cover every seat we can name today and a role editor is a permanent support liability. Inside a fleet the four seats map cleanly onto four humans in a small delivery company — the person who signed the contract, the person at the dispatch desk, the person who does the banking, and the person on the bike.

The permission codes are additive and there is no DENY in this namespace, unlike admin. The reason is that a fleet has no cross-tenant reach to revoke: every query is already narrowed to `fleet_org_id` by the `scoped()` repo wrapper before a permission is even consulted, so the permission check is only ever answering "which of this fleet's own screens may this seat open".

ROLE CODE	PERMISSIONS	WHAT THE SEAT IS FOR
FLEET_OWNER	All 15 fleet permissions	The signatory. Exactly one per org.
FLEET_DISPATCH	f.offer.*; f.job.read, f.job.assign, f.rider.manage, f.vehicle.manage, f.area.request	The control-room seat: accepts offers and allocates riders.
FLEET_FINANCE	f.payout.read, f.cod.declare, f.bank.manage, f.job.read	Declares COD and reconciles earnings.
FLEET_RIDER	f.job.read, f.job.manifest, f.job.scan, f.pod.write, f.job.fail	Mobile-only seat. Sees only jobs assigned to that rider, and only inside the window.

All 15 permissions in the fleet namespace

GROUP	KEY	WHAT IT ALLOWS	STEP-UP
Offers	f.offer.read	See live offers addressed to this fleet	—
Offers	f.offer.respond	Accept or decline an offer before it expires	—
Jobs	f.job.read	View this fleet's jobs and their masked drop details	—
Jobs	f.job.assign	Attach a rider and a vehicle to an accepted job	—
Jobs	f.job.manifest	Open the job manifest, which discloses the drop address	—
Jobs	f.job.scan	Record pickup and handover scans	—
Jobs	f.pod.write	Capture proof of delivery: OTP, geo point and optional photo	—
Jobs	f.job.fail	Record a failed attempt with an outcome code	—
Fleet	f.rider.manage	Add, edit, deactivate a rider; upload rider documents	—
Fleet	f.vehicle.manage	Add or retire a vehicle and update insurance expiry	—
Fleet	f.area.request	Request a service-area or rate-card change	—
Finance	f.payout.read	View earnings lines, runs and the fleet ledger	—
Finance	f.cod.declare	Declare cash collected and record the remittance reference	yes
Finance	f.bank.manage	Add or replace the payout bank account	yes
Org	f.user.manage	Invite, re-role or disable a user in this fleet	yes

The invite flow is deliberately identical to the supplier one described in AA-06 section 6.2, because a second invite implementation is a second place for a token-expiry bug to live. An admin creates the fleet org and invites exactly one FLEET_OWNER; that owner then invites everyone else from FL-24. Invite tokens are single-use and expire in 72 hours, the accept screen at FL-02 sets the password and accepts terms in one submit, and the owner and finance seats must enrol TOTP before the session is issued. There is no self-service company registration: a fleet cannot exist without a staff member having created it, which is the first and cheapest control over the least trusted actor class.

A FLEET_RIDER session is a much smaller thing than the other three, and the difference is enforced in four places rather than by hiding navigation. The rider logs in by mobile OTP instead of a password (same WhatsApp-first

channel as the customer app, so no new delivery path). The token carries the rider seat, so GET /fleet/jobs returns only jobs assigned to that rider_id rather than the whole org's board. The shell renders no desktop navigation at all for that role, so a rider who is handed a laptop sees the same single-column stack. And the five rider permissions omit every read of money, riders and org settings, so a stolen rider phone leaks a handful of masked job cards and nothing about the company. It does leak the addresses of that rider's currently open jobs, which is the residual risk and is accepted knowingly — see AA-08 section 8.6.

The 3 step-up gated actions, and why each one is gated

PERMISSION	ENDPOINT	WHY A FRESH FACTOR IS DEMANDED
f.cod.declare	POST /fleet/cod/:id/declare	This is the fleet asserting how much of the platform's cash it is holding. The declared figure is what admin reconciles a bank credit against, so a wrong or hostile declaration is the start of a money dispute rather than a UI mistake. requireFreshFactor(300) means the person typing the figure held the second factor within the last five minutes.
f.bank.manage	PUT /fleet/bank	Replacing the payout account redirects every future rupee the fleet earns. This is the single highest-value target in the namespace and the classic account-takeover payoff. The endpoint also pauses payouts until a staff member re-verifies the account, so a successful attacker gets a delay and a review rather than a transfer.
f.user.manage	POST /fleet/users	Inviting a user is privilege creation. Without a fresh factor, one stolen owner session could mint a second owner seat and survive the original password reset. Re-rolling an existing user runs through the same guard for the same reason.

Notice what is not step-up gated, because the omissions were argued. Accepting an offer is not gated, even though it commits the fleet to a job with a penalty attached: an offer lives for ten minutes and a TOTP prompt in the middle of that window would lose the race, which is a worse outcome than the risk. Opening the manifest is not gated either, and that is the more uncomfortable of the two. The reasoning is that a rider on a bike cannot produce an authenticator code one-handed, and a gate that people route around is a fiction; the control on the manifest is the disclosure register plus a four-hour window, not a second factor. AA-08 section 8.4 records this as an accepted deviation rather than an oversight, and it is the one decision in this document most likely to be revisited after the pilot.

7.3 THE 24 SCREENS

Complete inventory, grouped by the four jobs, with the permission that opens each one

This is the whole surface. 24 routes, no hidden ones, and every route in the table below is reachable from navigation or from a row action on another listed screen — there is no orphan page that only exists as a redirect target. The permission column is the single gate the router checks before mounting; a seat without it gets the 403 shell described in section 7.10, not a blank screen and not a disabled menu item.

One numbering caveat before the tables. The task lists in AA-11 refer to fleet screens by an earlier numbering (P6-FE2-02 says "FL-08 to FL-10" for the dispatcher board and P7-FE2-02 says "FL-18 to FL-22" for the money screens). The numbering in this section is the frozen one and AA-11 should be read against it: the dispatcher board is FL-06 and FL-07, and the money screens are FL-19 to FL-22. The work described in those tasks is unchanged; only the labels drifted.

Getting in — FL-01 to FL-03

Three screens before any work happens, and the only ones a fleet user sees on their first day. FL-01 is one screen with two modes rather than two screens, because a rider who lands on a password form will call the dispatcher rather than switch tabs. The mode is chosen by what the user types: an email address goes down the password path, a ten-digit mobile number goes down the OTP path.

ID	ROUTE	SCREEN	PERMISSION GATE	WHAT IT DOES
FL-01	/login	Sign in	public	Password for staff seats, mobile OTP for riders. One screen, two modes.
FL-02	/invite/token	Accept invitation	invite token	Set password, accept terms, TOTP for owner and finance.
FL-03	/onboarding	Onboarding checklist	session	Six steps: profile, KYC, insurance, bank, first service area, first rider.

The dispatch surface — FL-04 and FL-05

The two screens that matter. FL-04 is the home route for every seat except the rider, and it is a single page holding two things: the live offer strip with countdown rings, and the active-job board underneath it. They are on one screen because a dispatcher deciding whether to accept needs to see how loaded the fleet already is, and a two-click journey to find that out is how offers are lost. FL-05 is the same offer list at full width for a busy hour, and section 7.4 specifies it in detail.

ID	ROUTE	SCREEN	PERMISSION GATE	WHAT IT DOES
FL-04	/	Control room	f.offer.read	Live offer strip with countdowns above an active-job board. The dispatch home screen.
FL-05	/offers	Offer inbox	f.offer.read	Fee, weight, pincode, promised_by, seconds left. Accept or decline with a reason.

Job execution — FL-06 to FL-10

Once an offer is accepted the job is the fleet's problem and these five screens are how they discharge it. FL-06 is a board with columns by state and drag-onto-rider assignment, which is the one piece of interaction sugar in the portal; it earns its keep because assigning twenty jobs to four riders through a form is genuinely slow. FL-07 is masked by default with a Reveal address button that names the disclosure register in its own label, before it is pressed. FL-08, FL-09 and FL-10 are the three consequential writes: the manifest, the proof, and the failure.

ID	ROUTE	SCREEN	PERMISSION GATE	WHAT IT DOES
FL-06	/jobs	Job board	f.job.read	Columns by state. Drag a job onto a rider lane to assign.
FL-07	/jobs:id	Job detail	f.job.read	Masked view by default. A visible Reveal address button that logs the disclosure.
FL-08	/jobs:id/manifest	Delivery manifest	f.job.manifest	Full address, map link, receiver name. Window countdown in the header.
FL-09	/jobs:id/pod	Capture proof of delivery	f.pod.write	OTP entry, geolocation capture, optional photo, receiver name.
FL-10	/jobs:id/fail	Record failed attempt	f.job.fail	Outcome code, note, geo capture. Third attempt warns about forced RTO.

The rider stack — FL-11 and FL-12

Two screens, one column, 360 px. FL-11 shows one job card at a time in promised_by order with a single primary action per state, so there is never a moment where the rider must choose between two plausible next taps. FL-12 is the scan surface and it is the first casualty if the schedule slips: C-5 drops the camera and leaves manual code entry, which is slower at the supplier gate but loses no capability.

ID	ROUTE	SCREEN	PERMISSION GATE	WHAT IT DOES
FL-11	/rider	My deliveries	f.job.read	Rider-only stack: today's jobs in sequence, one card at a time.
FL-12	/rider/scan	Scan	f.job.scan	Camera scan for pickup and handover. Manual code entry fallback.

Riders and vehicles — FL-13 to FL-16

The fleet's own roster. The live job count against the hard cap of 8 is on the row rather than one click in, because that number is the only reason a dispatcher opens this screen mid-shift. Licence and insurance expiry both surface as amber chips 30 days out, fed by the doc-expiry-watch cron, and an expired document is what stops a rider being attachable rather than a separate compliance screen. FL-16 exists only if C-7 is not taken; Riders exist; vehicles do not. Heavy equipment goes to ATV-SELF.

ID	ROUTE	SCREEN	PERMISSION GATE	WHAT IT DOES
FL-13	/riders	Riders	f.rider.manage	Status, live job count out of 8, rating, licence expiry warning.
FL-14	/riders/new	Add rider	f.rider.manage	Name, mobile, licence number and expiry, document upload.
FL-15	/riders/:id	Rider detail	f.rider.manage	Job history, POD quality score, documents, leave toggle.
FL-16	/vehicles	Vehicles	f.vehicle.manage	Registration last four, kind, capacity, insurance expiry countdown.

Service areas — FL-17 and FL-18

Where a fleet says what it will carry and for how much. FL-18 accepts a bulk pincode paste because the realistic input is forty pincodes copied out of a spreadsheet, and a per-row form for that is a reason to give up. Nothing on these two screens takes effect on submit: an area change is a request that an admin approves (AA-05 section 5.6), and the screen shows a pending banner until it is decided. If C-8 is taken, FL-18 still submits but the approval is an admin editing the row directly, which works with three fleets and does not work with ten.

ID	ROUTE	SCREEN	PERMISSION GATE	WHAT IT DOES
FL-17	/areas	Service areas	f.area.request	Approved pincode list with the rate card, and a pending-request banner.
FL-18	/areas/request	Request area change	f.area.request	Bulk pincode paste, per-pincode fee, COD toggle, weight cap.

Money — FL-19 to FL-22

Four read-mostly screens and one consequential write. FL-19 and FL-20 itemise what the platform owes: base fee, distance, COD handling fee and penalties as separate lines, never netted into one opaque figure, because a netted figure is what turns a settlement question into an argument. FL-21 is the append-only ledger and carries the blocked-from-offers banner when the balance is negative. FL-22 is the declaration, and section 7.7 works through it in paise.

ID	ROUTE	SCREEN	PERMISSION GATE	WHAT IT DOES
FL-19	/earnings	Earnings	f.payout.read	Per-job base, distance, COD fee, penalties, net.
FL-20	/earnings/:runId	Payout run	f.payout.read	Line detail with UTR last four.
FL-21	/ledger	Ledger	f.payout.read	Append-only balance. A negative balance shows a blocked-from-offers banner.
FL-22	/cod	Cash on delivery	f.cod.declare	Open periods, expected cash, age buckets, declare action.

Settings — FL-23 and FL-24

The two screens with step-up guards on them. FL-23 shows the account as last four only the moment it is saved, including to the person who typed it, and states plainly that changing it pauses payouts until a staff member re-verifies. FL-24 is the team list; rider seats are created here and are visibly marked as mobile-only so nobody wastes a morning wondering why a rider cannot see the job board.

ID	ROUTE	SCREEN	PERMISSION GATE	WHAT IT DOES
FL-23	/settings/bank	Bank account	f.bank.manage	Last four only after save. Changing it pauses payouts.
FL-24	/settings/users	Team	f.user.manage	Dispatch, finance and rider seats. Rider seats get no desktop nav.

7.4 THE OFFER INBOX

Broadcast, then first accept wins. The screen the whole portal exists to serve

When a supplier marks a shipment `READY_FOR_PICKUP`, the API creates a `delivery_jobs` row and broadcasts it. Broadcast means every eligible fleet in the drop pincode receives a `job_offers` row with a TTL, sees the same card at the same moment, and the first one to accept owns the job. The rejected alternatives are worth naming because both are more sophisticated and both lose. Assigning to a single best fleet by score fails because with three fleets there is no score worth computing, and a single assignee who is at lunch costs the customer thirty-five minutes rather than ten seconds. A real auction where fleets bid a price fails because the rate card is already agreed per pincode in `fleet_service_areas`, so there is nothing to bid on, and because bidding turns a ten-second decision into a negotiation.

Eligibility for a wave is not a preference, it is four hard filters: the fleet serves the drop pincode, its `max_weight_g` covers the shipment, `cod_allowed` is true if the order is cash, and its `fleet_ledger` balance is not negative. The last filter is the important one — a fleet that has not remitted COD is excluded silently from every wave, which is the auto-block described in section 7.7 doing its work before any human is involved. Heavy equipment adds a fifth filter: the fleet must own a `PICKUP` or `TRUCK` vehicle with valid insurance, which is why C-7 taking away the vehicle registry routes heavy jobs to the internal `ATV-SELF` fleet by default.

WAVE	WHO IS IN IT	TTL	WHAT HAPPENS WHEN IT LAPSES
1	Tier-1 fleets serving the drop pincode, ordered by least-recently-offered.	600 s (10 min)	Every wave-1 offer goes <code>EXPIRED</code> and wave 2 opens. The job stays <code>OFFERED</code> throughout; only the offers change state.
2	Tier-1 plus tier-2 fleets in the pincode, same ordering, excluding anyone who declined.	600 s (10 min)	Same again, and wave 3 opens. A fleet that declined in wave 1 is not asked twice.
3	Every eligible fleet in the pincode regardless of tier, including <code>ATV-SELF</code> .	900 s (15 min)	The job transitions <code>OFFERED</code> to <code>UNASSIGNED</code> , raises an ops alert and appears on the admin exception board. RB-01 applies: rebroadcast at wave ALL, and if it is still unassigned twenty minutes later, assign <code>ATV-SELF</code> .

The three TTLs come from `OFFER_WAVE_TTL_SECONDS` and total 2100 seconds, so an unwanted job reaches `UNASSIGNED` in about 35 minutes. That figure is a floor, not a deadline, and AA-02 section 2.8 explains why: expiry is driven by the offer-sweep cron at `*/2 * * * *`, so a lapse is noticed up to two minutes after it happens. Worst case is therefore closer to thirty-seven minutes. The alternative — a per-offer timer or a queue with a visibility timeout — needs a process we do not have and a bill we cannot pay, so the portal is built to tolerate the granularity instead: the countdown ring on the card is computed from `expires_at` on the client, so a dispatcher never sees a live counter on an offer that is already dead in the database, and an accept that arrives after `expires_at` is refused by the same conditional `UPDATE` that refuses a loser.

What an offer card shows, and what it deliberately withholds

FIELD	PRE-ACCEPT	REASONING
Pickup area	shown	Locality and pincode of the supplier gate. Enough to know whether the leg starts where the fleet's riders already are.
Drop pincode	shown	Pincode only, with the locality name. This is the field a dispatcher actually decides on, and it is not personal data.
Distance band	shown	A band, not a figure: under 5 km, 5 to 15 km, over 15 km. A precise distance would be computed from the exact drop point, which is the thing being withheld.
Weight band	shown	From <code>weight_g</code> , banded. Determines whether a two-wheeler can take it, which is the second question after distance.
COD amount	shown	The exact cash the rider will carry, in rupees. Withholding this would be indefensible: it is the fleet's own cash risk and it changes who they send.
Payout	shown	<code>fee_quote_paise</code> , computed from that fleet's own rate card for that pincode. Each fleet sees its own number, and no fleet sees another's.
promised_by	shown	The delivery promise already made to the customer. A dispatcher declining because the window is too tight is a good outcome, not a lost job.
Customer name	withheld	Never shown at any point in the job lifecycle, before or after acceptance. The POD screen collects a receiver name; it does not display an expected one.
Street address	withheld	Released only after acceptance, only through the manifest, only inside the window, and only with a register row written. Section 7.5.

FIELD	PRE-ACCEPT	REASONING
Customer phone	withheld	Never released under any code path, to any fleet seat, under any permission. There is no endpoint that returns it and no permission that would unlock one.

WHY THE ADDRESS CANNOT BE SHOWN BEFORE ACCEPTANCE

This is the design constraint that shapes the whole screen, and it is arithmetic rather than principle. An offer is broadcast: in wave 3 every eligible fleet in the pincode holds a live offer for the same job at the same time. If the card carried the street address, then one customer's address would have been handed to every one of those companies, and all but one of them will never touch the parcel. With three fleets that is two unnecessary disclosures per job; with ten fleets it is nine, and the pii_disclosures register would fill with rows that record nothing useful because they cannot be tied to a delivery. Withholding the address until fleet_org_id is set is what makes the register meaningful: every row in it belongs to a company that actually carried the job. The cost is real and is paid at the dispatcher's desk — they accept on a pincode and a distance band, and occasionally accept something awkward. That is the trade, made knowingly.

The race, and exactly what the loser sees

Two dispatchers press Accept in the same millisecond. There is no lock, no queue and no advisory serialisation; there is one conditional UPDATE, and the database decides. Every accept also carries an Idempotency-Key under I-12, so a dispatcher whose phone retried the request gets the first response byte for byte rather than a second attempt at the race.

```
-- POST /fleet/offers/:id/accept (inside one transaction)
UPDATE delivery_jobs
  SET fleet_org_id = $fleet, status = 'ACCEPTED', accepted_at = now()
 WHERE id = $job
   AND status = 'OFFERED'
   AND fleet_org_id IS NULL
RETURNING id;

-- 1 row -> this fleet won. Mark its job_offers row ACCEPTED, mark every sibling
--          offer for the same job WITHDRAWN, in the same transaction. Commit.
-- 0 rows -> someone else won, or the sweep expired it. Roll back, 409 OFFER_ALREADY_TAKEN.
```

The loser's experience is specified, not left to the error boundary, because a dispatcher who loses a race and sees a red toast saying "Something went wrong" will stop trusting the inbox within a day. On 409 the client rolls back its optimistic state and replaces the card with a quiet, non-alarming "taken by another fleet" line. The card then disappears on the next poll, at most ten seconds later. No penalty is recorded, nothing is written to the fleet's acceptance-rate metric, and the loss is not counted as a decline — only an explicit decline with a reason code feeds that metric. Losing races is the expected steady state of a broadcast auction and must cost the fleet nothing at all.

The inbox polls GET /fleet/offers every 10 seconds. There was a server-sent events stream at GET /fleet/offers/stream and it is cut: C-1 removes it and falls back to polling everywhere, saving 2 dev-days. The reasoning in the cut register is that riders will not notice; dispatch sees a few seconds of extra latency. The endpoint stays in the AA-04 contract as a documented no-op so the client does not need a code change to adopt it later, and the polling interval is a constant in one place. Ten seconds against a ten-minute TTL is a 1.7 % worst-case loss of decision time, which is not the thing that will cost a fleet a job.

7.5 THE JOB LIFECYCLE AND THE MANIFEST

Six fleet-driven transitions, and the one endpoint that releases a customer address

delivery_jobs has 13 states and a transition table enforced by assertTransition under I-05; the note on the table definition in AA-03 says eleven, which predates the RTO pair and should be read as this table. Six transitions belong to the fleet and are the only ones this portal can cause. Everything else is system (the allocator and the sweep) or staff (manual assignment and cancellation), and no fleet screen offers a control for them. A dispatcher who wants a cancellation calls support, because a fleet cancelling a job it has already accepted is exactly the abuse the state machine exists to prevent.

STATE	MAY MOVE TO	DRIVEN BY	NOTE
DRAFT	OFFERED	system	Created with the shipment. No fleet attached.

STATE	MAY MOVE TO	DRIVEN BY	NOTE
OFFERED	ACCEPTED, UNASSIGNED	system	One or more job_offers live. First accept wins; the rest are WITHDRAWN in the same transaction.
UNASSIGNED	OFFERED, ASSIGNED_MANUAL	system, staff	All waves exhausted. Sits on the admin exception board.
ASSIGNED_MANUAL	ACCEPTED	staff	Ops assigned a fleet directly, bypassing the auction.
ACCEPTED	PICKED_UP, CANCELLED	fleet	Fleet owns the job. A rider must be attached before PICKED_UP.
PICKED_UP	IN_TRANSIT, CANCELLED	fleet	Scan at the supplier gate.
IN_TRANSIT	OUT_FOR_DELIVERY, RTO_INITIATED	fleet	Moving.
OUT_FOR_DELIVERY	DELIVERED, FAILED_ATTEMPT	fleet	Manifest unmask window is open.
FAILED_ATTEMPT	OUT_FOR_DELIVERY, RTO_INITIATED	fleet	Requires a pod_records row with an outcome.
DELIVERED	— terminal	fleet	Requires OTP match and a POD inside the 200 m geofence.
RTO_INITIATED	RTO_DELIVERED, CANCELLED	fleet, staff	Reverse leg.
RTO_DELIVERED	— terminal	fleet	Confirmed back with the supplier.
CANCELLED	— terminal	staff	Only staff may cancel a job already accepted.

The dispatcher drives ACCEPTED to PICKED_UP to IN_TRANSIT to OUT_FOR_DELIVERY, and then either DELIVERED or FAILED_ATTEMPT, plus the RTO leg if two reattempts are exhausted. Two of those have preconditions the UI must state before the button is pressed rather than after: PICKED_UP requires a rider attached and the shipment already at READY_FOR_PICKUP, and it takes the supplier gate code — a wrong code is 409 GATE_CODE_MISMATCH, rate limited to five attempts per job, which is a deliberate brake on guessing. DELIVERED requires a pod_records row that passed both the OTP and the geofence. Attaching a rider is refused with 409 RIDER_AT_CAPACITY at the cap of 8 concurrent jobs, and the FL-06 board shows the live count on the rider lane so the dispatcher sees the wall before hitting it.

Transitioning to OUT_FOR_DELIVERY is the consequential one. It generates the six-digit delivery OTP, stores only its bcrypt hash on the job, sends the digits to the customer over a WhatsApp template, and opens the manifest window. Everything in the rest of this section hangs off that moment.

GET /fleet/jobs/:id/manifest — the only PII exception in the programme

I-06 is absolute across all three portals: last four characters only, no reveal affordance, and there is deliberately no pii.read.raw permission in the Permission type union for any role including SUPER_ADMIN. This one endpoint is the single exception in the entire programme, and it exists because a parcel cannot be delivered to a masked address. It is worth being precise about how narrow the exception is, because every one of these constraints is load-bearing.

CONSTRAINT	WHAT IT MEANS IN CODE
One code path	The manifest handler is the only caller of unmaskAddress(), which is exported from packages/shared/pii.ts but not from the barrel. A grep for _cipher outside that file and this handler returns nothing, and that grep is a CI gate (P2-BE1-04).
Address only	The response carries address lines, locality, pincode, coordinates for the map link and the receiver name. It carries no phone, no email, no order value, no basket contents and no customer identity beyond the name the parcel is addressed to.
Job-scoped	The :id must resolve to a job whose fleet_org_id equals the token claim, through scoped(). Another fleet's job id returns 404, not 403, so the endpoint cannot be used to probe whether a job exists.
State-gated	Permitted only while the job is ACCEPTED, PICKED_UP, IN_TRANSIT or OUT_FOR_DELIVERY. Before acceptance there is no fleet to disclose to; after DELIVERED or RTO_DELIVERED the need has ended.
Window-scoped	MANIFEST_WINDOW_MINUTES = 240, i.e. 4 hours. Outside it the endpoint returns 403 DISCLOSURE_WINDOW_CLOSED and writes no register row, because a refused read is not a disclosure.
Logged in the same transaction	The read is wrapped by discloseOrThrow(), which appends a pii_disclosures row with actor_kind fleet, the fleet_user_id, the subject customer, field address, reason_code DELIVERY_MANIFEST and the job as ref. Same transaction, so a disclosure without a register row is not a state the database can hold.
Read-only forever	pii_disclosures rows are retained 24 months and are never purged by a job. No screen in the admin portal can delete one; AD-38 is a read-only register with filters.

```
GET /api/v1/fleet/jobs/9f21.../manifest      Authorization: Bearer <atv_fleet>
200 OK
{
  "job_code": "DJ-2026-004182",
  "receiver_name": "S. Kulkarni",
  "address": { "line1": "12, Sadashiv Peth", "line2": "Near Vitthal Mandir",
               "locality": "Sadashiv Peth", "city": "Pune", "pincode": "411030" },
  "geo": { "lat": 18.5089, "lng": 73.8478 },
  "window_expires_at": "2026-08-30T18:42:11Z",
  "cod_amount_paise": 48750,
  "disclosure_id": "pd_01J9..."          <- the register row written for THIS read
}

403 DISCLOSURE_WINDOW_CLOSED  after the window. No body fields, no register row.
404 NOT_FOUND                 another fleet's job id. Indistinguishable from a bad id.

There is no phone field. Not masked, not last-four, not null. Absent.
```

AN AMBIGUITY IN THE WINDOW THAT MUST BE SETTLED BEFORE W6 STARTS

The two sources disagree on when the 240-minute clock starts. The environment table describes MANIFEST_WINDOW_MINUTES as how long an unmask stays valid **after OUT_FOR_DELIVERY**, and the job state machine says the window is open at OUT_FOR_DELIVERY. Task P6-BE1-01 says the gate is **MANIFEST_WINDOW_MINUTES since acceptance**. These are different endpoints, and taken literally the first reading leaves an unbounded window: a job sitting in ACCEPTED for eight hours passes the state gate with no clock running against it at all. The resolution to implement, and the one this document assumes, is the conjunction of both: the state must be one of the four, *and* now() minus COALESCE(out_for_delivery_at, accepted_at) must be inside the window. That closes the hole without loosening either source. AA-04 section 4.6 should be amended to state it in one sentence before BE-1 writes the handler, because a disagreement discovered during implementation will be resolved by whoever is typing.

Now the operational consequence, which is the part that will actually generate support tickets. A rider is standing in a lane holding a parcel, has the address, and cannot find the door. Under R-04 this is rated high impact and it is an accepted gap rather than a solved problem. The rider cannot call the customer, because the phone number does not exist anywhere in this portal. What they do instead is ring the platform support line; ops opens the job in AD-20, and triggers a WhatsApp template to the customer from the job screen — the template asks the customer to call the support line or share a landmark, and it never contains the rider's number either. RB-03 is the runbook and its last clause is the important one: trigger the template, record a failed attempt if it comes to that, and do not reveal the number. Ops has no unmask affordance of its own either — a staff member who needs the address to resolve a dispute goes through a step-up authorised action that writes a SUPPORT_ESCALATION row in the same register.

The honest version of that paragraph is that it inserts a human into the loop at the worst possible moment, and it works only because the pilot has two suppliers and one fleet. The named upgrade is a masked call bridge — Exotel or Kaleyra — at roughly ₹0.60 per minute plus a ₹1 500 per month platform fee. That is a recurring cost, so under I-14 it needs a written budget waiver naming the amount, the trigger and the fallback; the trigger to write it is the first week in which more than two deliveries fail for reachability. Nothing about the portal changes when the waiver is signed: the manifest gains a Call button that dials a bridge number, and still shows no digits.

7.6 PROOF OF DELIVERY

Three pieces of evidence, none of which the rider can fabricate alone

FL-09 is the screen that closes a job, and it is designed on the assumption that the person filling it in may be trying to close a job they did not complete. That is not cynicism about riders; it is the only defensible position when the record produced here is the platform's entire case in a return dispute weeks later. Three pieces of evidence are captured, and the point of having three is that each one covers a different lie.

EVIDENCE	HOW IT IS CHECKED	WHAT IT PROVES	WHAT IT CANNOT PROVE
Delivery OTP	Six digits typed by the rider, compared against otp_hash on the job with bcrypt. The digits are never stored, never returned in a body and never written to a log line or a Sentry breadcrumb.	That a person holding the customer's phone was present and consented to the handover.	That the right parcel was handed over, or that its contents were intact. Only the geo point and the photo speak to that.

EVIDENCE	HOW IT IS CHECKED	WHAT IT PROVES	WHAT IT CANNOT PROVE
Geo point	One navigator.geolocation reading with its accuracy, compared against the drop coordinates. <code>POD_GEOFENCE_METRES = 200</code> ; <code>distance_m</code> is persisted whether it passes or not.	That the handover happened at approximately the right door, at the recorded time.	Anything at all indoors or in a high-rise, where a 200 m reading is optimistic. This is why it flags rather than blocks.
Photo	Uploaded straight to Cloudinary by the browser using a job-scoped signature valid ten minutes. The Cloudinary secret never leaves the API.	The physical state of the parcel and the doorstep at handover — the only evidence a damage claim can be argued against.	Who received it. Faces are not required and <code>receiver_name</code> is typed, not verified.

The rider never sees the OTP. It is generated at `OUT_FOR_DELIVERY`, hashed immediately, and the digits travel to the customer over the WhatsApp template. There is no resend button in the fleet portal, because a fleet-triggered resend is functionally a fleet-readable OTP after two attempts; a resend is an ops action from AD-20. A rider who says the customer never received it is describing a support ticket, not a UI gap.

ON OTP MISMATCH

The response is 422 `OTP_MISMATCH` with an attempts-remaining count. Five attempts are allowed per delivery attempt, matched to the gate-code limit for consistency, after which the OTP path is locked for that attempt and the only remaining actions on the screen are Record failed attempt (FL-10) and Call support. Locking rather than escalating is deliberate: six wrong digits is far more often a customer at the wrong address in the same building than an attack, and a lockout that leaves the rider a documented exit is cheaper than one that leaves them stranded. The failed attempt writes a `pod_records` row with outcome `CUSTOMER_UNAVAILABLE`, which counts toward the two-reattempt limit.

OUTSIDE THE GEOFENCE

The first submission beyond 200 m returns 422 `OUT_OF_GEOFENCE` with the measured distance, so a rider who is at the wrong gate of a large complex can walk and retry. If they confirm with a reason code, the POD is accepted and flagged: `distance_m` is persisted, the job reaches `DELIVERED`, and the record lands on the admin exception board under RB-04 with the fleet's payout line held pending an explanation within 24 hours. Hard-blocking would be wrong because by that point the parcel is already in the customer's hands; refusing the record does not undo the handover, it just creates a delivered parcel with no delivery, which is strictly worse evidence than a flagged one.

Note the divergence this resolves. AA-04 section 4.5 states the endpoint returns 422 `OUT_OF_GEOFENCE`, and RB-04 in AA-09 describes flagging the POD and holding the payout — which only makes sense if the POD was recorded. Both are correct under the two-step behaviour above: the plain call is refused, the confirmed call is accepted and flagged. The contract test should assert both paths, because a single test asserting only the 422 will pass against an implementation that can never record a flagged POD at all, and that gap would not surface until the first dispute.

The photo is the cut. C-6 drops POD photo capture for 1 dev-day, leaving OTP plus geofence, and the register is blunt about the cost: Weakens dispute evidence. Take this cut only for equipment-free, low-value routes. It also has a second trigger that is not about the schedule — RB-10 says that if Cloudinary credits cross 17 in a month, POD photo capture is dropped for the rest of that month and client compression is tightened. That makes C-6 the only cut in the programme that can be taken by an operational threshold rather than a planning decision, so the client must treat the photo as optional from the first commit: a feature flag read at runtime, never a build constant, and a POD form that is valid with the photo field empty.

7.7 COD AND SETTLEMENT

The section most likely to cause a real argument with a fleet

A fleet employee takes notes from a customer's hand and walks away with them. Everything else in this document is a software problem; this is a cash-handling problem with a portal attached to it, and it is the one place where the platform and the fleet have directly opposed incentives. R-09 rates cash going missing between rider and platform as high impact and high likelihood, and the detection signal is named: variance above 2 % of expected in any weekly period. The design response is not a report someone reads on Monday — it is an automatic block that removes the fleet from the auction before a human is involved.

A `cod_remittances` row is created per fleet per period by the `cod-period-open` cron at 05:00 on Mondays, summing collected COD from `pod_records` for that fleet. The FSM below is the shipped one, per P7-BE2-04. The status enum recorded against the table definition in AA-03 is longer — it lists `OPEN`, `DECLARED`, `IN_TRANSIT`, `RECEIVED`, `RECONCILED`, `SHORT` and `DISPUTED` — and that is the older, wider draft. Five states are being built. `DISPUTED` is deliberately not one of them: a dispute is a state of a conversation, and modelling it here would put a status on the money row that nobody can define the exit conditions for. A disputed remittance stays `SHORT` and is tracked on the exception board.

STATE	MOVED BY	WHAT IT MEANS	WHAT THE FLEET SEES ON FL-22
DUE	system	The period is closed and expected_paise is fixed. Nothing has been declared.	The open period with the expected cash, an age counter in days, and a Declare button.
DECLARED	fleet (step-up)	The fleet has asserted declared_paise and recorded a deposit reference. This is a claim, not a receipt.	The declared figure, the reference last four, and a waiting-for-confirmation state. Still blocked if the balance is negative.
DEPOSITED	fleet	The bank transfer has been made and the reference is attached. A provisional ledger credit is posted here so a fleet that has genuinely paid is not blocked over a weekend.	Deposited, awaiting reconciliation. The blocked banner clears at this point if nothing else holds it.
RECONCILED	admin (step-up)	A staff member matched received_paise against the bank statement and variance_paise is zero or within tolerance. Terminal.	Closed, with the three figures shown side by side. No further action.
SHORT	admin (step-up)	received_paise is below expected_paise. A compensating debit is posted; the original row is never edited. Terminal for this period, and the shortfall carries in the ledger.	Short by an exact amount, the compensating debit itemised, and — above the threshold — a prepaid-only notice with the figure needed to clear it.

The variance calculation, worked in paise

Every figure below is an integer in a BIGINT column named *_paise, per I-04. There is no float anywhere in this path, including in the client: FL-19 to FL-22 format from paise for display and never parse a rupee string back. Variance is measured against expected, not against declared, because the number that matters is what reached the bank versus what the customers paid — a fleet that declares honestly and deposits short is in exactly the same position as one that declares short.

Fleet FL-0002, period 2026-08-17 to 2026-08-23, 14 COD deliveries.

expected_paise	486750	sum(cod_amount_paise) over DELIVERED COD jobs	= Rs 4,867.50
declared_paise	486750	the fleet claims the full amount	= Rs 4,867.50
received_paise	483000	what the bank statement actually shows	= Rs 4,830.00

variance_paise = received_paise - expected_paise = 483000 - 486750 = -3750 = Rs -37.50
variance_bps = 3750 * 10000 / 486750 = 77.04 -> 77 bps (integer division, floored)

77 bps < COD_VARIANCE_BLOCK_BPS (200) -> status SHORT, compensating debit of 3750 paise,
NO move to prepaid-only. Ops asks for an explanation.

Same period, alternative outcome:

received_paise	476000	= Rs 4,760.00
variance_paise = 476000 - 486750	= -10750	= Rs -107.50
variance_bps = 10750 * 10000 / 486750	= 220.9 -> 220 bps	

220 bps > 200 -> status SHORT, compensating debit of 10750 paise, fleet blocked from offers,
and moved to prepaid-only orders until the period is reconciled (RB-05).

Two details in that arithmetic are worth stating because they will be argued. The bps figure is floored by integer division and the comparison is strictly greater than the threshold, so a variance measuring 200.9 bps floors to 200 and does not block, while 201.0 bps does; the boundary is defined in one place and the test fixture pins both sides of it. And the threshold is checked against the period, not cumulatively, so a fleet that is 150 bps short three weeks running never trips the automatic block. That is a real hole. The mitigation is the second trigger rather than a cleverer formula: any negative fleet_ledger balance blocks the fleet regardless of variance, and three consecutive shortfalls will produce one, because each compensating debit stays on the ledger until it is paid.

The fleet ledger, the payout run, and why a reversal is a new row

fleet_ledger is append-only and protected by a Postgres rule that raises on UPDATE and DELETE, proven in CI. The balance shown on FL-21 is recomputed from the rows on every read rather than stored, so the number in the portal and the number a nightly assertion computes cannot silently diverge. The same ledger holds both what the platform owes the fleet and the COD cash the fleet is holding, which is why one balance can answer the only question that matters to the allocator: may this fleet be offered work.

fleet_ledger, FL-0002, same period	amount_paise	balance_after_paise	
COD_HELD 14 COD jobs delivered	-486750	-486750	fleet holds the cash
JOB_EARNING base + distance + COD fee	+112000	-374750	BLOCKED: balance < 0
COD_DEPOSIT provisional, at DEPOSITED	+486750	+112000	unblocked
COD_SHORTFALL compensating, at SHORT	-3750	+108250	the 3750 paise gap
PAYOUT_RELEASED run PR-2026-W34	-108250	0	paid out, UTR ...4471

Nothing above was ever UPDATED. The COD_SHORTFALL row does not amend COD_DEPOSIT; both are visible, and FL-21 renders the pair adjacent so the fleet reads the story rather than a net figure.

Earnings accrue per delivery event, not per period: the accrual cron at 02:30 posts one fleet_payouts line per job delivered the previous day, computed from that fleet's own rate card — base_fee_paise, per_km_paise times the distance, and cod_fee_bps applied to the collected amount. Penalties and RTO fees are separate signed rows and are never folded into an adjusted credit, because a fleet that sees one number it cannot reconstruct will ask for the breakdown by phone, and the answer must already be on the screen. The payout-draft cron assembles the DRAFT run at 04:00 on Mondays; membership freezes when a staff member approves it, and release is a step-up gated admin action (AA-05 section 5.7). FL-20 shows the run with the UTR as last four only, including in any export.

When something must be reversed — a clawback on a returned shipment, a penalty applied late, a shortfall recovered — it is a new signed row referencing the original, and both stay visible. This is not bookkeeping fastidiousness; it is what makes RB-07 answerable. A fleet disputing a figure is shown the rows, and a row that was silently edited is a dispute the platform cannot win. Wallet cash-out does not exist anywhere in the programme, for fleets or customers, because holding and redeeming stored value needs an RBI PPI licence the programme does not have.

7.8 THE RIDER SURFACE

A responsive web view at 360 px. Not an app, and honest about what that costs

The rider gets two routes, FL-11 and FL-12, out of the same Vite build and the same Cloudflare Pages deployment as the dispatcher screens. There is no native application, no store listing and no review cycle. The reason is capacity rather than conviction: FE-2 has ten weeks and two portals, and a React Native rider app would consume the second portal entirely. The reason it is defensible is that the rider's whole interaction is four taps on a job card and one form, and a web view does four taps and a form perfectly well when it loads. The reason it is uncomfortable is everything in the table below.

WHAT IS MISSING	WHY	WHAT IT COSTS IN PRACTICE
Background location	A browser cannot read location with the tab backgrounded, and continuous ingest is a named paid gap regardless of the client.	No live tracking for the customer, no automatic arrival detection, and the geo point exists only because the rider has the POD form open in the foreground when they submit it.
A general offline write queue	Queueing state transitions offline means reconciling a stale FSM against a server that has moved, which is a fortnight of work and a class of bug we cannot afford to ship.	Every transition needs connectivity at the moment it is tapped. The one exception is the POD photo, which has a narrow in-memory retry queue so a network drop mid-upload loses nothing (P9-FE2-01).
Push notifications	FCM in a browser on a mid-range Android is unreliable: it depends on the vendor's battery policy and is routinely killed. A notification channel that silently fails is worse than none.	A rider does not learn about a new assignment until they look. Dispatchers phone their riders, which is what they already do.
Camera scan (C-5)	Manual shipment-code entry only. Saves 2 dev-days.	Manual shipment-code entry at the supplier gate. Slower and more error-prone, but the gate code is rate limited to five attempts so a typo is recoverable.
Vehicle registry (C-7)	Riders exist; vehicles do not. Heavy equipment goes to ATV-SELF. Saves 3 dev-days.	The heavy-equipment eligibility filter cannot be satisfied by any third-party fleet, so those jobs route to the internal ATV-SELF fleet by default. Third-party heavy delivery is blocked until it lands.
An installable app	No store presence, no icon on the home screen unless the rider adds it manually.	The rider reaches the portal through a browser bookmark or a link the dispatcher sends. Session length is 14 days, so a rider logs in roughly twice a month rather than daily.

THE FAILURE MODE THIS CREATES, AND THE ONLY MITIGATION ACTUALLY AVAILABLE

A rider is in a basement car park or a rural stretch with one bar. They have the parcel, the customer has the OTP, the handover happens, and POST /fleet/jobs/:id/pod cannot reach the API. Nothing in the portal can queue that write, so the job stays OUT_FOR_DELIVERY while the parcel is gone. The mitigation is three things and none of them is elegant. First, the manifest is fetched once and held in memory, so the rider who has already opened it can still find the door without connectivity — losing signal after the manifest loads does not strand them. Second, the POD form holds its state locally and retries on reconnect, so walking twenty metres to the street usually completes it; the photo has a retry queue and the OTP and geo fields survive the round trip. Third, if it still fails, the dispatcher can see the job stuck OUT_FOR_DELIVERY on FL-06 and calls the rider, and ops can close it from the admin side with a staff-recorded POD that is flagged as such. The honest summary is that a delivery in a dead zone degrades into a phone call, and the programme accepts that at pilot scale with one fleet.

The performance bar is set in AA-11 as a gate rather than an aspiration: the fleet job screen must be interactive within four seconds on a 3G-throttled mid-range Android browser (P9-FE2-01). That single number does more for the rider surface than any feature would, because every failure described above gets worse the longer the screen takes to become usable.

7.9 WHAT A FLEET CAN NEVER SEE OR DO

Each row names the mechanism, because an intention without a mechanism is a wish

The rows below are not a summary of good behaviour; they are the list a reviewer should try to break. Every one names the thing in the code that stops it, and where that thing is a test or a CI gate rather than a runtime check, the row says so.

A FLEET CANNOT	WHAT IT GETS INSTEAD	ENFORCING MECHANISM
See a customer phone number, at any point, in any role	Nothing. There is no masked variant on any fleet screen.	No fleet endpoint selects the phone columns. The Prisma extension strips every *_cipher field unconditionally, and unmaskAddress() — the only escape hatch — does not touch phone at all. The ten-consecutive-digit grep over every captured fleet response fixture is a CI gate (P9-BE1-01).
See a customer name	A receiver name on the manifest, which is the name the parcel is addressed to.	The masked job payload has no name field. The POD form collects receiver_name as free text and does not display an expected value to compare against.
See an address before its offer is accepted	Pickup locality, drop pincode and a distance band.	GET /fleet/offers has no address field in the contract. The address lives behind a separate endpoint that requires fleet_org_id on the job to equal the token claim, which is NULL until accept commits.
See an address after the window closes	403 DISCLOSURE_WINDOW_CLOSED and no register row.	The manifest handler gates on state plus 240 minutes before it calls unmaskAddress(). A refused read writes nothing, so the register counts disclosures and not attempts.
Re-read the manifest of a delivered job	The masked job record, indefinitely. Pincode and locality only.	DELIVERED and RTO_DELIVERED are outside the permitted state set, so a completed job's address is not readable again by the fleet that delivered it — not from history, not from an export, not from FL-07. This is the row that is easiest to forget when building the job-history screen.
See another fleet's offers, acceptance rate or payout	Its own numbers only.	Every query goes through scoped(), which injects fleet_org_id from the verified claim. A body or query string carrying fleet_org_id is refused with 400 SCOPE_IN_REQUEST by the aa/no-tenant-in-input rule and the guard behind it, before any handler runs.
See an order or shipment it has no job for	The job, and the shipment fields the job needs.	There is no /fleet/orders route at all. Shipment reads are joined through delivery_jobs, and a job id belonging to another fleet returns 404 rather than 403 so existence cannot be probed.
Write to any ledger	A declaration, which is a claim an admin confirms.	No fleet endpoint writes fleet_ledger. Accrual is a cron; credits and debits are posted by the money module. A Postgres rule raises on UPDATE and DELETE against the ledger tables regardless of caller.
Reach the admin or partner namespace	401 ACTOR_MISMATCH.	requireActor() compares the aud claim against the mounted namespace immediately after JWT verification, before any handler. The twelve wrong-combination test in P1-BE1-02 covers every pair.
Cancel a job it has accepted, or move an order status	A support call, and a failed-attempt or RTO path it can drive itself.	CANCELLED is a staff-only transition in the job FSM and assertTransition refuses it from a fleet principal. orders.status is derived by reduceOrderStatus() and has no setter outside orders/fsm.ts.

7.10 STATES, OFFLINE REALITY AND ACCESSIBILITY

Four states per screen, a 360 px floor, and what happens when a poll fails mid-shift

Every screen in this portal implements the same four states as the other two portals, with one variation that only exists here and in the supplier portal. The variation is that a cross-tenant read returns 404 rather than 403: a fleet asking for a job belonging to another fleet is told the job does not exist, because 403 would confirm that it does. This is the difference between a permission failure and a tenancy failure, and conflating them turns any id in a URL into an enumeration oracle.

STATE	WHEN	WHAT THE SCREEN DOES
Loading	First fetch, or a refetch with no cached data.	Skeleton rows at the real row height so the layout does not jump. No spinner over a populated table — a refetch with cached data keeps the old rows and marks them stale instead.
Empty	The query succeeded and returned nothing.	A sentence naming why it is empty and the one action that would change it. An empty offer inbox says the fleet is eligible and waiting, or names the block if it is blocked, because those two look identical and mean opposite things.
Error	Network failure, 5xx, or an unexpected shape.	The last good data with a staleness banner where data exists; a retry affordance and the request id where it does not. Never a bare error string, and never a screen that loses the rider's current job.
Forbidden	A permitted route, a forbidden action, or a foreign tenant id.	403 renders a shell explaining which permission is missing and who in the fleet can grant it. A foreign tenant id renders the ordinary not-found page, indistinguishable from a mistyped id.

The poll failing mid-shift is the case worth specifying precisely, because it is the one that happens every day. GET /fleet/offers is polled every ten seconds and GET /fleet/jobs on a similar cadence. When a poll fails, the client keeps the last good response, shows a small staleness banner with the age of the data, and continues polling with backoff. Two rules follow from that and both are testable. An accepted job never disappears from the board because of a failed poll — the cache is authoritative until a successful response replaces it, so a dispatcher who loses connectivity keeps a correct picture of work in progress rather than watching their jobs vanish. And an offer card whose countdown reaches zero is removed by the client on the countdown, not by the poll, so a stale list cannot invite a dispatcher to accept something that expired four minutes ago.

The accessibility floor for a one-handed phone interaction

The rider surface is used standing up, in sunlight, one-handed, sometimes wearing a glove, by someone who is not reading carefully. That is a harder target than the dispatcher desktop and it sets the floor for the whole portal rather than being a separate rider standard. Concretely: a 360 px viewport with no horizontal scroll at any point, primary actions at least 44 by 44 px and positioned in the lower half of the screen where a thumb reaches, one primary action per state so there is never a choice between two plausible next taps, and text contrast at 4.5:1 minimum measured against the actual background rather than the design token. Every input on the POD form has a real label, not a placeholder, because a placeholder disappears the moment the rider starts typing the OTP. The numeric fields use inputmode numeric so the phone shows a keypad. And no state is communicated by colour alone — a flagged POD and a clean one differ by a word, not a chip colour, which also happens to be what makes them legible in sunlight. The axe and keyboard pass in P9-FE2-01 holds this portal to the same bar as admin, and the four-second interactive budget on throttled 3G is part of the same gate.

7.11 WHAT THIS PORTAL WILL NOT DO IN THESE TEN WEEKS

Seven omissions, each with its workaround and the point at which it stops working

Five of these are entries in the AA-00 cut register and two are paid gaps that cannot be cut because they were never in. The distinction matters: a cut is dev-days we chose not to spend and can spend later, while a paid gap is a recurring bill that needs a written waiver against I-14 naming the amount, the trigger and the fallback. Nothing in this list is a surprise to discover in week nine.

OMISSION	WHAT HAPPENS INSTEAD	WHERE IT BREAKS
C-1 · Fleet SSE offer stream	Fall back to 10 s polling everywhere. The endpoint stays in the contract as a documented no-op.	Nowhere at pilot volume. It starts to matter when a fleet runs more than about ten dispatchers on one account and the polling load becomes visible on Cloud Run, or when TTLs are shortened below five minutes and ten seconds of latency becomes a real fraction of the decision window.

OMISSION	WHAT HAPPENS INSTEAD	WHERE IT BREAKS
C-5 · Rider mobile scan camera	Manual shipment-code entry only.	At the supplier gate with more than roughly twenty parcels in one handover, where typing codes is both slow and the point at which mistyped codes start generating GATE_CODE_MISMATCH support calls.
C-6 · POD photo capture	OTP plus geofence only; no photo. Can also be triggered mid-month by RB-10 when Cloudinary credits cross 17.	At the first damage claim on a high-value or equipment delivery. The register is explicit: take this cut only for equipment-free, low-value routes, and it must be a runtime flag rather than a build constant so the operational trigger can use it.
C-7 · Vehicle registry	Riders exist; vehicles do not. Heavy equipment goes to ATV-SELF.	The moment a third party wants to carry heavy equipment. Until then ATV-SELF absorbs it, which caps equipment volume at what staff can physically deliver — the real ceiling, and it is low.
C-8 · Service-area approval workflow	Admin edits fleet areas directly.	Fine with three fleets. Does not scale past ten. Past ten fleets an admin editing rate cards by hand is both a bottleneck and an unaudited money change, which is the worse half of that sentence.
Paid gap · masked click-to-call between rider and customer	A time-boxed address reveal on the manifest, plus an ops-triggered WhatsApp template to the customer. The rider never gets a number and never gives one.	The first week with more than two deliveries failed for reachability. The named upgrade is Exotel or Kaleyra at roughly ₹0.60 per minute plus ₹1 500 per month, and it needs a written I-14 waiver.
Paid gap · rider GPS tracking on a map for the customer	Status milestones only — HANDED_OVER, IN_TRANSIT, OUT_FOR_DELIVERY — and one geo point captured at POD, which is all the geofence and the dispute record need.	When customers begin asking where the rider is often enough to generate support volume. The cost is Maps Platform beyond the \$200 monthly credit plus a location ingest path, so it is the most expensive item on this list and the least likely to be bought.

THE ONE THING ON THIS LIST THAT IS NOT REALLY OPTIONAL

Read R-02 alongside the table above. The programme's largest fleet risk is not any missing feature — it is that no third-party delivery company onboards in time for the pilot at all, with the trigger being fewer than two fleet orgs at KYC_VERIFIED by the end of W8. The fallback is that the internal ATV-SELF fleet org, seeded in migration M5 as a tier-1 fleet over the current serviceable pincodes, does the deliveries and staff use this portal to do them. That is why every screen specified here is built even if the external fleet count is zero, and why go-live is not delayed for onboarding. It also means the first real user of this portal will almost certainly be a colleague, which is the best possible outcome for finding out what section 7.8 got wrong.

Where the work lands

The build order for these 24 screens is fixed in AA-11 and is worth stating here because it follows from section 7.1 rather than from convenience. W2 delivers authentication and onboarding, FL-01 to FL-03, because nothing can be tested against a portal nobody can log into. W5 delivers the offer surface and the allocator behind it — FL-04, FL-05, the wave engine, the expiry sweep and the job board — which is the week that decides whether this product works. W6 delivers execution and proof: the manifest with its disclosure logging, the OTP path, the geofence and the failed-attempt tree. W7 is money: earnings, the ledger, the COD declaration and the auto-block. W9 is the accessibility and resilience pass, including the throttled-3G budget and the race hammering that proves exactly one winner per offer across a thousand iterations. Read this document alongside AA-04 sections 4.5 and 4.6 for the endpoint contracts, AA-05 for what happens on the admin side of every escalation named here, AA-08 for the disclosure regime in full, and AA-09 for the five runbooks that will be opened in anger — RB-01, RB-03, RB-04, RB-05 and RB-10.